MASTERING

Java · Selenium · TestNG · Jenkins · BDD with Cucumber · REST Assured API · GITHUB

& Test Automation Framework Development

A Complete Guide for QA Engineers & Automation Professionals

– Ranjit Appukutti

Covers:

Java 21/25 (LTS) | Selenium WebDriver 4.41+ | TestNG 7.12+ | Jenkins CI/CD
Page Object Model | Data-Driven Testing | BDD with Cucumber | REST Assured API Testing
Selenium Grid 4 + Docker | CDP / DevTools Protocol | Allure Reports | GitHub Actions

Chapter 1: Introduction to Software Testing & Automation

1.1 What is Software Testing?

Software testing is the process of evaluating and verifying that a software application or system does what it is supposed to do. It involves executing a program with the intent of finding bugs, ensuring the system meets the defined requirements, and validating the quality of the product.

Testing is not merely about finding defects , it also validates that new features work as expected, existing functionality is not broken by changes, performance meets acceptable thresholds, and the user experience is smooth and reliable.

1.1.1 Types of Testing

Testing Type	Description	Example Tools
Unit Testing (Developers)	Testing individual units or components in isolation	JUnit, TestNG, Mockito
Integration Testing (QA's)	Testing combined units working together	TestNG, REST Assured
System Testing (QA's)	End-to-end testing of the complete system	Selenium, Cypress
Regression Testing (QA's)	Verifying existing functionality after changes	Selenium, TestNG Suites
Performance Testing (QA's)	Validating speed, stability, scalability	JMeter, Gatling
UAT (Business/QA's)	End-user acceptance validation	Manual + Automated

1.2 Manual vs Automated Testing

Manual testing involves human testers executing test cases step by step without automated tools. While valuable for exploratory testing and usability assessment. Manual testing can be time-consuming and error-prone when repeated frequently..

Test automation uses software tools to execute pre-scripted tests automatically, comparing actual outcomes against expected ones. Automation excels at regression testing, load testing, and repetitive scenarios.

> **IMPORTANT**
> Automation does not replace manual testing it augments it. Reserve automation for stable, repetitive test cases and manual effort for exploratory and usability testing.

When to Automate

- Tests that are executed repeatedly (regression suite)
- Tests requiring multiple data inputs (data-driven)
- Tests on multiple browsers and operating systems
- Smoke and sanity tests before each release
- Performance and load tests with many concurrent users

When NOT to Automate

- Test cases that change frequently
- One-time or rarely executed tests
- Exploratory testing and ad-hoc scenarios
- Tests requiring visual/UX judgment

1.3 Overview of the Technology Stack

This book teaches you to build professional test automation frameworks using a proven, industry-standard technology stack:

Technology	Role in Framework	Version (Book)
Java	Core programming language	Java 21 / Java 25 (LTS)
Selenium WebDriver	Browser automation engine	4.41+
TestNG	Test runner & reporting framework	7.12+
Maven	Build and dependency management	3.9.x
Jenkins	CI/CD pipeline and scheduling	2.4xx LTS
ExtentReports	Rich HTML test reports	5.x
Log4j2	Logging framework	2.23+
Apache POI	Excel data-driven testing	5.3+

Chapter 2: Core Java for Test Automation

Java is the most widely used programming language for enterprise test automation. Its object-oriented nature, vast ecosystem, and strong typing make it ideal for building large, maintainable test frameworks. This chapter covers Java fundamentals specifically applied to automation engineering.

2.1 Java Development Environment Setup

Installing Java JDK

1. Download JDK 21 (or JDK 25 LTS)
2. Run the installer and note the installation path
3. Set JAVA_HOME environment variable to the JDK root directory
4. Add %JAVA_HOME%\bin to your PATH
5. Verify installation by running: java -version

```
java -version
javac -version
```

Setting Up IntelliJ IDEA

- Download IntelliJ IDEA Community Edition (free) from jetbrains.com/idea
- Install and configure JDK 21 as project SDK (JDK 25 also supported)
- Install plugins: Maven Helper, TestNG, CheckStyle

> ✓ **PRO TIP**
> Use IntelliJ IDEA over Eclipse for new projects. It has superior refactoring tools, better Maven integration, and a more intuitive UI.

2.2 Java OOP Concepts for Automation

2.2.1 Classes and Objects

In test automation, classes represent pages, components, utilities, and test suites. Understanding class design is critical for maintainable frameworks.

```
public class LoginPage {
    private WebDriver driver;
    private By usernameField = By.id("username");
    private By passwordField = By.id("password");
    private By loginButton = By.id("loginBtn");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }
```

```
    public void enterUsername(String username) {
        driver.findElement(usernameField).sendKeys(username);
    }

    public void clickLogin() {
        driver.findElement(loginButton).click();
    }
}
```

2.2.2 Inheritance

Inheritance allows test classes to share common setup and teardown code through a base class:

```
public class BaseTest {
    protected WebDriver driver;

    @BeforeMethod
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts()
            .implicitlyWait(Duration.ofSeconds(10));
    }

    @AfterMethod
    public void tearDown() {
        if (driver != null) driver.quit();
    }
}

public class LoginTest extends BaseTest {
    @Test
    public void testSuccessfulLogin() {
        // driver is already initialized from BaseTest
        LoginPage loginPage = new LoginPage(driver);
        loginPage.enterUsername("admin");
        loginPage.enterPassword("pass123");
        loginPage.clickLogin();
    }
}
```

2.2.3 Interfaces and Abstractions

```
public interface PageInterface {
```

```
    boolean isPageLoaded();
    String getPageTitle();
}

public class DashboardPage implements PageInterface {
    private WebDriver driver;
    @Override
    public boolean isPageLoaded() {
        return driver.getCurrentUrl().contains("/dashboard");
    }
    @Override
    public String getPageTitle() {
        return driver.getTitle();
    }
}
```

2.3 Collections in Test Automation

Java collections are extensively used in automation for managing test data, storing web elements, and handling configuration.

Key Collections Used in Automation

Collection	Common Use Case	Example
List<T>	Storing multiple WebElements	List<WebElement> links = driver.findElements(By.tagName("a"))
Map<K,V>	Test data key-value pairs	Map<String, String> testData = new HashMap<>()
Set<T>	Unique values (no duplicates)	Set<String> windowHandles = driver.getWindowHandles()
Queue<T>	Sequential test execution order	Queue<String> testQueue = new LinkedList<>()

2.4 Exception Handling in Test Automation

Robust exception handling prevents test frameworks from crashing unexpectedly and provides meaningful error messages:

```
public WebElement findElementSafely(By locator) {
    try {
        return driver.findElement(locator);
    } catch (NoSuchElementException e) {
        logger.error("Element not found: " + locator);
        throw new RuntimeException("Element not found: " + locator, e);
```

```
    } catch (StaleElementReferenceException e) {
        logger.warn("Stale element, retrying...");
        return driver.findElement(locator); // Retry once
    } catch (TimeoutException e) {
        logger.error("Timeout waiting for element: " + locator);
        throw new RuntimeException("Timeout: " + locator, e);
    }
}
```

2.5 Java 8+ Features in Test Automation

Lambda Expressions

```
// Without Lambda
List<WebElement> visibleElements = new ArrayList<>();
for (WebElement el : allElements) {
    if (el.isDisplayed()) visibleElements.add(el);
}

// With Lambda (Stream API)
List<WebElement> visibleElements = allElements.stream()
    .filter(WebElement::isDisplayed)
    .collect(Collectors.toList());
```

Optional for Null Safety

```
Optional<WebElement> element = driver.findElements(locator)
    .stream().findFirst();
element.ifPresent(el -> el.click());
String text = element.map(WebElement::getText).orElse("Not Found");
```

> ✓ **PRO TIP**
> Use Streams and Lambdas to write cleaner, more readable test utilities. They are especially useful when working with lists of WebElements.

Chapter 3: Selenium WebDriver — Complete Guide

Selenium WebDriver is the industry-standard tool for automating web browser interactions. It provides a programming interface to create and run browser tests against web applications. Selenium 4 introduced significant improvements including the W3C WebDriver standard compliance, improved APIs, and Chrome DevTools Protocol (CDP) support.

3.1 Selenium Architecture

Selenium WebDriver operates through a client-server architecture:

- **Selenium WebDriver operates through a client–server architecture:**
- **Test Script (Java)** – Your automation code that calls WebDriver APIs
- **WebDriver Client Library** – Language bindings that translate automation commands into the **W3C WebDriver protocol**
- **Browser Driver** – Browser-specific executable (ChromeDriver, GeckoDriver, EdgeDriver) that implements the W3C protocol.

> **ℹ SELENIUM 4 KEY CHANGE**
> Selenium 4 uses the W3C WebDriver standard instead of the legacy JSON Wire Protocol. This means ChromeDriver and GeckoDriver communicate natively without the need for a Selenium Server for local execution.

3.2 Setting Up Selenium with Maven

Maven Dependencies (pom.xml)

```
<dependencies>
  <!-- Selenium Java -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.41.0</version>
  </dependency>

  <!-- WebDriverManager - auto browser driver management -->
  <dependency>
    <groupId>io.github.bonigarcia</groupId>
    <artifactId>webdrivermanager</artifactId>
    <version>5.9.2</version>
  </dependency>

  <!-- TestNG -->
```

```
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.12.0</version>
  </dependency>
</dependencies>
```

3.3 WebDriver Initialization

Chrome Browser

```
// Using WebDriverManager (recommended)
WebDriverManager.chromedriver().setup();
ChromeOptions options = new ChromeOptions();
options.addArguments("--start-maximized");
options.addArguments("--disable-notifications");
options.addArguments("--incognito");
WebDriver driver = new ChromeDriver(options);
```

Firefox Browser

```
WebDriverManager.firefoxdriver().setup();
FirefoxOptions options = new FirefoxOptions();
WebDriver driver = new FirefoxDriver(options);
```

Headless Chrome (for CI/CD)

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--headless=new");
options.addArguments("--no-sandbox");
options.addArguments("--disable-dev-shm-usage");
options.addArguments("--window-size=1920,1080");
WebDriver driver = new ChromeDriver(options);
```

> ✓ **PRO TIP**
> Headless mode is recommended for CI/CD pipelines as it is faster and does not require a display server. However, some scenarios may require headed execution using virtual display tools like Xvfb.

3.4 Locating Web Elements

Finding elements is the foundation of Selenium automation. Selenium 4 uses the By class for all locator strategies:

Locator Strategy	Code Example	Best Use Case
By.id()	By.id("username")	Fastest — use when IDs are present
By.name()	By.name("email")	Form fields with name attributes
By.className()	By.className("btn-primary")	Elements with unique CSS classes
By.cssSelector()	By.cssSelector("#form .input")	Complex CSS selections
By.xpath()	By.xpath("//input[@type='text']")	Complex hierarchical selections
By.linkText()	By.linkText("Sign In")	Anchor tags with exact text
By.tagName()	By.tagName("table")	Finding all elements of a type

XPath vs CSS Selector

CSS Selectors are generally faster, but XPATHs are more flexible. Prefer ID or CSS selectors when available because they are more readable and usually perform better than complex XPath expressions.

```
// CSS Selector - preferred for most cases
By.cssSelector("#login-form input[name='username']")
By.cssSelector(".nav-menu > li:first-child a")

// XPath - use when CSS can't do it
By.xpath("//button[contains(text(),'Submit')]")
By.xpath("//label[text()='Email']/following-sibling::input")
By.xpath("//td[text()='John']/parent::tr/td[2]")
```

3.5 WebDriver Actions — Complete Reference

Navigation

```
driver.get("https://example.com");          // Navigate to URL
driver.navigate().to("https://example.com"); // Navigate
driver.navigate().back();                    // Browser back
driver.navigate().forward();                 // Browser forward
driver.navigate().refresh();                 // Refresh page
driver.getCurrentUrl();                      // Get current URL
driver.getTitle();                           // Get page title
```

Element Interactions

```
WebElement element = driver.findElement(By.id("username"));
```

```
element.click();                    // Click element
element.sendKeys("John Doe");       // Type text
element.clear();                    // Clear input field
element.getText();                  // Get element text
element.getAttribute("value");      // Get attribute
element.isDisplayed();              // Check visibility
element.isEnabled();                // Check if enabled
element.isSelected();               // Check if selected (checkbox)
```

Dropdown (Select)

```
Select dropdown = new Select(driver.findElement(By.id("country")));
dropdown.selectByVisibleText("India");
dropdown.selectByValue("IN");
dropdown.selectByIndex(2);
dropdown.getFirstSelectedOption().getText();
```

Actions Class — Advanced Interactions

```
Actions actions = new Actions(driver);

// Mouse hover
actions.moveToElement(menuItem).perform();

// Drag and drop
actions.dragAndDrop(source, target).perform();

// Double click
actions.doubleClick(element).perform();

// Right click (context menu)
actions.contextClick(element).perform();

// Key combinations
actions.keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).perform();
```

3.6 Waits in Selenium

Synchronization is one of the most critical aspects of reliable Selenium automation. Incorrect waits lead to either flaky tests (too short) or slow tests (too long).

Types of Waits

- Implicit Wait: Sets a global timeout for finding elements

- Explicit Wait: Waits for a specific condition before proceeding
- Fluent Wait: Advanced explicit wait with polling interval and ignored exceptions

Implicit Wait (Avoid in Modern Frameworks)

```
// Sets global timeout for all findElement calls
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

> **WARNING**
> Do not mix implicit and explicit waits — this causes unpredictable wait times. Modern frameworks should use explicit waits exclusively.

Explicit Wait (Recommended)

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));

// Wait for element to be visible
WebElement element = wait.until(
    ExpectedConditions.visibilityOfElementLocated(By.id("btn")));

// Wait for element to be clickable
wait.until(ExpectedConditions.elementToBeClickable(By.id("submit")));

// Wait for URL to contain a string
wait.until(ExpectedConditions.urlContains("/dashboard"));

// Wait for text to appear in element
wait.until(ExpectedConditions.textToBePresentInElement(msgEl, "Success"));
```

Fluent Wait (Most Flexible)

```
Wait<WebDriver> fluentWait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))
    .pollingEvery(Duration.ofMillis(500))
    .ignoring(NoSuchElementException.class)
    .ignoring(StaleElementReferenceException.class);

WebElement element = fluentWait.until(d -> d.findElement(By.id("dynamic")));
```

3.7 Handling Special Scenarios

Alerts and Popups

```
// Switch to alert and interact
Alert alert = driver.switchTo().alert();
```

```
alert.accept();           // Click OK
alert.dismiss();          // Click Cancel
alert.getText();          // Get alert message
alert.sendKeys("text");   // Enter text in prompt
```

Frames and iFrames

```
// Switch to frame by index, name, or element
driver.switchTo().frame(0);
driver.switchTo().frame("frameName");
driver.switchTo().frame(driver.findElement(By.id("myFrame")));

// Return to main document
driver.switchTo().defaultContent();
```

Multiple Windows/Tabs

```
String mainWindow = driver.getWindowHandle();
Set<String> allWindows = driver.getWindowHandles();

for (String windowHandle : allWindows) {
    if (!windowHandle.equals(mainWindow)) {
        driver.switchTo().window(windowHandle);
        // Interact with new window
        driver.close(); // Close popup
    }
}
driver.switchTo().window(mainWindow); // Back to main
```

JavaScript Executor

```
JavascriptExecutor js = (JavascriptExecutor) driver;

// Scroll to element
js.executeScript("arguments[0].scrollIntoView(true);", element);

// Click element (bypass overlays)
js.executeScript("arguments[0].click();", element);

// Scroll to bottom of page
js.executeScript("window.scrollTo(0, document.body.scrollHeight);");

// Get hidden element text
String text = (String) js.executeScript(
```

```
    "return arguments[0].innerText;", element);
```

> ✅ **PRO TIP**
> Use JavascriptExecutor as a last resort. If a test needs JS to click an element, investigate why the normal click fails first. JS clicks bypass actual user interactions.

3.8 Taking Screenshots

```
public static void captureScreenshot(WebDriver driver, String testName) {
    TakesScreenshot ts = (TakesScreenshot) driver;
    File srcFile = ts.getScreenshotAs(OutputType.FILE);
    String path = "./screenshots/" + testName + "_"
        + System.currentTimeMillis() + ".png";
    try {
        FileUtils.copyFile(srcFile, new File(path));
        System.out.println("Screenshot saved: " + path);
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Chapter 4: TestNG — Test Framework Mastery

TestNG (Test Next Generation) is a powerful testing framework inspired by JUnit but with more advanced features. It supports test configuration, grouping, parallel execution, data providers, and rich reporting — making it the preferred choice for enterprise Selenium automation frameworks.

4.1 TestNG Architecture and Annotations

TestNG uses annotations to define test methods, configure test execution order, and manage test lifecycle. Understanding these annotations is fundamental to building structured test suites.

Annotation	Execution Trigger	Common Use Case
@BeforeSuite	Once before all tests in suite	Start application server, initialize reporting
@AfterSuite	Once after all tests in suite	Generate final report, cleanup resources
@BeforeClass	Once before first @Test in class	Initialize class-level objects
@AfterClass	Once after last @Test in class	Release class-level resources
@BeforeMethod	Before each @Test method	Launch browser, navigate to URL
@AfterMethod	After each @Test method	Close browser, capture screenshot on fail
@BeforeGroups	Before specified groups run	Prepare test data for a group
@AfterGroups	After specified groups run	Cleanup group-specific data
@Test	Test method marker	All test case methods
@DataProvider	Provides test data to @Test	Data-driven testing with multiple datasets
@Factory	Creates test class instances	Dynamic test class creation
@Parameters	Pass XML parameters to tests	Environment/browser configuration

4.2 TestNG Configuration — testng.xml

The testng.xml file is the configuration backbone of TestNG. It controls which tests run, execution order, parallel execution, parameters, and groupings.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Regression Suite" parallel="methods" thread-count="4">

  <!-- Suite-level parameters -->
  <parameter name="browser" value="chrome"/>
  <parameter name="baseUrl" value="https://staging.example.com"/>

  <test name="Login Tests">
    <groups>
      <run>
        <include name="smoke"/>
        <include name="regression"/>
        <exclude name="skip"/>
      </run>
    </groups>
    <classes>
      <class name="tests.LoginTest">
        <methods>
          <include name="testValidLogin"/>
          <include name="testInvalidLogin"/>
        </methods>
      </class>
      <class name="tests.DashboardTest"/>
    </classes>
  </test>

  <test name="Checkout Tests">
    <classes>
      <class name="tests.CartTest"/>
      <class name="tests.PaymentTest"/>
    </classes>
  </test>
</suite>
```

4.3 Grouping Tests

Groups allow you to categorize tests and run specific categories selectively. This is essential for CI/CD pipelines where you may want to run only smoke tests on every commit.

```
@Test(groups = {"smoke", "regression"}, priority = 1)
public void testPageLoad() {
    Assert.assertEquals(driver.getTitle(), "Home Page");
}

@Test(groups = {"regression"}, priority = 2,
```

```
     dependsOnMethods = {"testPageLoad"})
public void testLoginForm() {
    // This runs only if testPageLoad passes
}

@Test(groups = {"skip"})
public void testWorkInProgress() {
    // This test will be excluded
}
```

4.4 Data-Driven Testing with @DataProvider

DataProviders allow the same test method to run multiple times with different data sets — a core pattern in test automation:

```
@DataProvider(name = "loginCredentials")
public Object[][] loginData() {
    return new Object[][] {
        {"admin", "admin123", true},    // valid login
        {"user", "wrong", false},         // invalid password
        {"", "admin123", false},          // empty username
        {"admin", "", false}              // empty password
    };
}

@Test(dataProvider = "loginCredentials")
public void testLogin(String username, String password,
                      boolean expectedSuccess) {
    LoginPage loginPage = new LoginPage(driver);
    loginPage.login(username, password);
    Assert.assertEquals(loginPage.isLoggedIn(), expectedSuccess,
        String.format("Login failed for user: %s", username));
}
```

4.5 Parallel Test Execution

Parallel execution dramatically reduces test suite execution time. TestNG supports parallelism at suite, test, class, and method levels:

```
<!-- Run test methods in parallel using 4 threads -->
<suite name="Parallel Suite" parallel="methods" thread-count="4">

<!-- Run test classes in parallel -->
<suite name="Parallel Suite" parallel="classes" thread-count="3">
```

```
<!-- Run test tags/groups in parallel -->
<suite name="Parallel Suite" parallel="tests" thread-count="2">
```

ℹ CRITICAL FOR PARALLEL EXECUTION

When running tests in parallel, use ThreadLocal<WebDriver> to ensure each thread has its own browser instance. Never share a single WebDriver across threads — this causes race conditions and test failures.

```
public class DriverManager {
    private static ThreadLocal<WebDriver> tlDriver
        = new ThreadLocal<>();

    public static WebDriver getDriver() {
        return tlDriver.get();
    }

    public static void setDriver(WebDriver driver) {
        tlDriver.set(driver);
    }

    public static void removeDriver() {
        tlDriver.remove(); // Prevent memory leaks
    }
}
```

4.6 TestNG Assertions

Assertions are the heart of test validation. TestNG's Assert class provides methods to verify expected vs actual outcomes:

```
// Hard Assertions - test stops on failure
Assert.assertEquals(actual, expected, "Error message");
Assert.assertNotEquals(value1, value2);
Assert.assertTrue(condition, "Should be true");
Assert.assertFalse(condition, "Should be false");
Assert.assertNull(object, "Object should be null");
Assert.assertNotNull(object, "Object should not be null");

// Soft Assertions - test continues despite failures
SoftAssert softAssert = new SoftAssert();
softAssert.assertEquals(pageTitle, "Dashboard");
softAssert.assertTrue(welcomeMsg.isDisplayed());
softAssert.assertEquals(userEmail, "test@example.com");
```

```
softAssert.assertAll(); // Reports all failures at once
```

PRO TIP

Use SoftAssert when you want to validate multiple conditions in a single test and report all failures at once, rather than stopping at the first failure.

4.7 TestNG Listeners

Listeners allow you to hook into TestNG's test execution lifecycle to add custom behavior like screenshot capture, reporting, and logging:

```
public class TestListener implements ITestListener {

    @Override
    public void onTestStart(ITestResult result) {
        System.out.println("Starting: " + result.getName());
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        System.out.println("PASSED: " + result.getName());
    }

    @Override
    public void onTestFailure(ITestResult result) {
        // Capture screenshot on failure
        WebDriver driver = DriverManager.getDriver();
        ScreenshotUtil.capture(driver, result.getName());
        System.out.println("FAILED: " + result.getName());
    }

    @Override
    public void onTestSkipped(ITestResult result) {
        System.out.println("SKIPPED: " + result.getName());
    }
}

// Register listener in testng.xml
<suite name="Suite">
  <listeners>
    <listener class-name="listeners.TestListener"/>
  </listeners>
</suite>
```

Chapter 5: Page Object Model & Framework Architecture

The Page Object Model (POM) is the most important design pattern in Selenium test automation. It creates a clean separation between test logic and page interaction code, making tests more maintainable, readable, and scalable.

5.1 What is Page Object Model?

In POM, each web page or significant UI component in the application is represented as a Java class. The class encapsulates the locators and interactions for that page, while test classes only call the methods exposed by the page objects.

Benefits of POM

- Maintainability: When UI changes, only the page class needs updating — not all tests
- Reusability: Page methods can be reused across multiple test classes
- Readability: Tests read like user stories rather than technical code
- Separation of Concerns: Locators are separate from test logic
- Reduced Duplication: No repeated findElement() calls across tests

5.2 Basic POM Implementation

Page Class

```
public class LoginPage {
    private WebDriver driver;
    private WebDriverWait wait;

    // Locators — private to the page
    private final By usernameInput = By.id("username");
    private final By passwordInput = By.id("password");
    private final By loginButton = By.cssSelector(".btn-login");
    private final By errorMessage = By.cssSelector(".error-msg");
    private final By forgotPasswordLink = By.linkText("Forgot Password?");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(15));
    }

    // Actions — public methods
    public void enterUsername(String username) {
        wait.until(ExpectedConditions.visibilityOfElementLocated(usernameInput))
```

```
            .sendKeys(username);
    }

    public void enterPassword(String password) {
        driver.findElement(passwordInput).sendKeys(password);
    }

    public DashboardPage clickLogin() {
        driver.findElement(loginButton).click();
        return new DashboardPage(driver); // Returns next page
    }

    public LoginPage login(String user, String pass) {
        enterUsername(user);
        enterPassword(pass);
        clickLogin();
        return this;
    }

    public String getErrorMessage() {
        return driver.findElement(errorMessage).getText();
    }
}
```

Test Class Using POM

```
public class LoginTest extends BaseTest {
    @Test(groups = {"smoke", "regression"})
    public void testSuccessfulLogin() {
        LoginPage loginPage = new LoginPage(driver);
        DashboardPage dashboard = loginPage.login("admin", "Admin@123");
        Assert.assertTrue(dashboard.isWelcomeDisplayed(),
            "Dashboard should be displayed after login");
    }

    @Test(groups = {"regression"})
    public void testInvalidPassword() {
        LoginPage loginPage = new LoginPage(driver);
        loginPage.login("admin", "wrongpassword");
        Assert.assertEquals(loginPage.getErrorMessage(),
            "Invalid credentials", "Error message mismatch");
    }
}
```

5.3 PageFactory — Enhanced POM

Selenium's PageFactory class provides the @FindBy annotation for cleaner locator declaration and lazy element initialization.

Note: While PageFactory simplifies locator initialization, many modern frameworks prefer explicit locators with WebDriverWait for better control and debugging.

```
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;

public class LoginPage {
    private WebDriver driver;

    @FindBy(id = "username")
    private WebElement usernameInput;

    @FindBy(id = "password")
    private WebElement passwordInput;

    @FindBy(css = ".btn-login")
    private WebElement loginButton;

    @FindBy(css = ".error-message")
    private WebElement errorMessage;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this); // Initialize all @FindBy
    }

    public void login(String username, String password) {
        usernameInput.sendKeys(username);
        passwordInput.sendKeys(password);
        loginButton.click();
    }
}
```

5.4 Complete Framework Structure

A professional Selenium framework follows a layered architecture. Here is the recommended project structure:

```
project-root/
├── src/
```

```
│   ├── main/java/
│   │   ├── base/
│   │   │   └── BasePage.java           # Base class for all pages
│   │   │   └── BaseTest.java           # Base class for all tests
│   │   ├── pages/
│   │   │   └── LoginPage.java
│   │   │   └── DashboardPage.java
│   │   │   └── CartPage.java
│   │   ├── utils/
│   │   │   └── DriverFactory.java      # WebDriver creation
│   │   │   └── WaitUtils.java          # Custom wait utilities
│   │   │   └── ScreenshotUtils.java    # Screenshot capture
│   │   │   └── ExcelUtils.java         # Excel data reading
│   │   │   └── ConfigReader.java       # Properties file reader
│   │   ├── listeners/
│   │   │   └── TestListener.java       # ITestListener impl
│   │   ├── constants/
│   │   │   └── Constants.java          # URL, timeout constants
│   ├── test/java/
│   │   ├── tests/
│   │   │   └── LoginTest.java
│   │   │   └── DashboardTest.java
│   │   ├── dataproviders/
│   │   │   └── LoginDataProvider.java
├── src/test/resources/
│   ├── testng.xml                      # TestNG suite config
│   ├── config.properties               # Environment config
│   └── testdata/                       # Excel/CSV test data
├── logs/                               # Log4j2 log files
├── screenshots/                        # Failure screenshots
├── reports/                            # ExtentReports output
└── pom.xml                             # Maven config
```

5.5 BasePage and BaseTest Classes

BasePage — Shared Page Utilities

```
public abstract class BasePage {
    protected WebDriver driver;
    protected WebDriverWait wait;
    protected Actions actions;

    public BasePage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(20));
```

```
        this.actions = new Actions(driver);
    }

    protected WebElement waitForVisible(By locator) {
        return wait.until(
            ExpectedConditions.visibilityOfElementLocated(locator));
    }

    protected void click(By locator) {
        waitForVisible(locator).click();
    }

    protected void type(By locator, String text) {
        WebElement el = waitForVisible(locator);
        el.clear();
        el.sendKeys(text);
    }

    protected String getText(By locator) {
        return waitForVisible(locator).getText();
    }

    protected boolean isElementPresent(By locator) {
        return !driver.findElements(locator).isEmpty();
    }
}
```

BaseTest — Test Lifecycle Management

```
public class BaseTest {
    protected WebDriver driver;
    private static final Logger logger
        = LogManager.getLogger(BaseTest.class);

    @Parameters({"browser", "baseUrl"})
    @BeforeMethod
    public void setUp(
            @Optional("chrome") String browser,
            @Optional("https://example.com") String baseUrl) {
        driver = DriverFactory.createDriver(browser);
        DriverManager.setDriver(driver);
        driver.get(baseUrl);
        logger.info("Browser: {} | URL: {}", browser, baseUrl);
    }
```

```
    @AfterMethod(alwaysRun = true)
    public void tearDown(ITestResult result) {
        if (result.getStatus() == ITestResult.FAILURE) {
            ScreenshotUtils.capture(driver, result.getName());
        }
        DriverManager.removeDriver();
        if (driver != null) driver.quit();
    }
}
```

Chapter 6: Advanced Framework Features

6.1 Data-Driven Testing with Apache POI (Excel)

Enterprise automation frameworks read test data from Excel files using Apache POI, enabling non-technical team members to manage test data without touching code.

Maven Dependency

```
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.3.0</version>
</dependency>
```

Excel Utility Class

```
public class ExcelUtils {
    private XSSFWorkbook workbook;
    private XSSFSheet sheet;

    public ExcelUtils(String filePath, String sheetName) {
        try (FileInputStream fis = new FileInputStream(filePath)) {
            workbook = new XSSFWorkbook(fis);
            sheet = workbook.getSheet(sheetName);
        } catch (IOException e) {
            throw new RuntimeException("Cannot open Excel: " + filePath);
        }
    }

    public int getRowCount() {
        return sheet.getLastRowNum() + 1;
    }

    public String getCellData(int rowNum, int colNum) {
        XSSFRow row = sheet.getRow(rowNum);
        if (row == null) return "";
        XSSFCell cell = row.getCell(colNum);
        if (cell == null) return "";
        DataFormatter df = new DataFormatter();
        return df.formatCellValue(cell);
    }

    public Object[][] getAllData() {
```

```
        int rows = getRowCount() - 1; // Exclude header
        int cols = sheet.getRow(0).getLastCellNum();
        Object[][] data = new Object[rows][cols];
        for (int r = 1; r <= rows; r++) {
            for (int c = 0; c < cols; c++) {
                data[r-1][c] = getCellData(r, c);
            }
        }
        return data;
    }
}
```

6.2 Configuration Management

config.properties

```
# Browser configuration
browser=chrome
headless=false

# Application URLs
base.url=https://staging.yourapp.com
api.base.url=https://api.staging.yourapp.com

# Timeouts (seconds)
implicit.wait=0
explicit.wait=20
page.load.timeout=30

# Test data
testdata.path=src/test/resources/testdata/TestData.xlsx

# Reporting
report.path=./reports
screenshot.path=./screenshots
```

ConfigReader Utility

```
public class ConfigReader {
    private static Properties props;
    private static final String CONFIG_PATH
        = "src/test/resources/config.properties";

    static {
```

```
        props = new Properties();
        try (FileInputStream fis = new FileInputStream(CONFIG_PATH)) {
            props.load(fis);
        } catch (IOException e) {
            throw new RuntimeException("Config file not found!");
        }
    }

    public static String get(String key) {
        String value = System.getProperty(key, props.getProperty(key));
        if (value == null) throw new RuntimeException("Key not found: " + key);
        return value;
    }

    public static int getInt(String key) {
        return Integer.parseInt(get(key));
    }

    public static boolean getBoolean(String key) {
        return Boolean.parseBoolean(get(key));
    }
}
```

ENVIRONMENT OVERRIDE

ConfigReader checks System.getProperty first, then falls back to config.properties. This allows overriding configurations from Maven command line: mvn test -Dbrowser=firefox -Dbase.url=https://prod.example.com

6.3 ExtentReports — Professional HTML Reports

Maven Dependency

```
<dependency>
  <groupId>com.aventstack</groupId>
  <artifactId>extentreports</artifactId>
  <version>5.1.1</version>
</dependency>
```

ExtentReport Manager

```
public class ExtentReportManager {
    private static ExtentReports extent;
    private static ThreadLocal<ExtentTest> test = new ThreadLocal<>();
```

```
    public static ExtentReports getInstance() {
        if (extent == null) {
            ExtentSparkReporter reporter = new ExtentSparkReporter(
                ConfigReader.get("report.path") + "/report.html");
            reporter.config().setDocumentTitle("Automation Report");
            reporter.config().setReportName("Regression Suite");
            reporter.config().setTheme(Theme.DARK);
            extent = new ExtentReports();
            extent.attachReporter(reporter);
            extent.setSystemInfo("OS", System.getProperty("os.name"));
            extent.setSystemInfo("Java", System.getProperty("java.version"));
            extent.setSystemInfo("Browser",
                ConfigReader.get("browser").toUpperCase());
        }
        return extent;
    }

    public static ExtentTest getTest() { return test.get(); }
    public static void setTest(ExtentTest t) { test.set(t); }
}
```

6.4 Log4j2 Logging

log4j2.xml Configuration

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{HH:mm:ss} %-5level [%t] %c{1} - %msg%n"/>
    </Console>
    <RollingFile name="FileAppender"
                 fileName="logs/automation.log"
                 filePattern="logs/automation-%d{yyyy-MM-dd}.log">
      <PatternLayout pattern="%d{ISO8601} %-5level %c{1} - %msg%n"/>
      <Policies>
        <TimeBasedTriggeringPolicy/>
      </Policies>
    </RollingFile>
  </Appenders>
  <Loggers>
    <Root level="INFO">
      <AppenderRef ref="Console"/>
      <AppenderRef ref="FileAppender"/>
    </Root>
```

```
  </Loggers>
</Configuration>
```

Using Logger in Test Code

```
public class LoginPage extends BasePage {
    private static final Logger log
        = LogManager.getLogger(LoginPage.class);

    public DashboardPage login(String user, String pass) {
        log.info("Logging in as user: {}", user);
        type(usernameInput, user);
        type(passwordInput, pass);
        log.debug("Clicking login button");
        click(loginButton);
        log.info("Login action complete");
        return new DashboardPage(driver);
    }
}
```

Chapter 7: Jenkins & CI/CD Pipeline Integration

Jenkins is the most widely used open-source automation server for building CI/CD pipelines. Integrating your Selenium test suite with Jenkins allows tests to run automatically on every code commit, providing immediate feedback on application quality.

7.1 Jenkins Architecture

- Jenkins Master: Orchestrates the pipeline, schedules jobs, and coordinates agents
- Jenkins Agent/Node: Executes the actual build and test commands
- Job/Pipeline: Configured workflow defining steps to build, test, and deploy
- Workspace: Isolated directory on the agent where job files are stored
- Build Trigger: Event that initiates a pipeline (commit, schedule, manual)

7.2 Installing Jenkins

6. Download Jenkins LTS WAR from jenkins.io/download
7. Run: java -jar jenkins.war --httpPort=8080
8. Navigate to http://localhost:8080 and enter the initial admin password
9. Install suggested plugins (includes Git, Maven, Pipeline)
10. Create first admin user

Required Plugins for Selenium Integration

- Maven Integration Plugin
- Git Plugin / GitHub Plugin
- HTML Publisher Plugin (for ExtentReports)
- Allure Jenkins Plugin (optional)
- Email Extension Plugin (for notifications)
- TestNG Results Plugin

7.3 Creating a Jenkins Pipeline (Jenkinsfile)

Declarative Pipelines defined in a Jenkinsfile stored in your source code repository are the modern, recommended approach. This enables Pipeline-as-Code:

```
pipeline {
    agent any

    environment {
        BROWSER = 'chrome'
```

```
        BASE_URL = 'https://staging.example.com'
        MAVEN_OPTS = '-Xmx1024m'
    }

    tools {
        maven 'Maven-3.9'
        jdk 'JDK-21'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/org/automation-repo.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean compile -q'
            }
        }

        stage('Run Smoke Tests') {
            steps {
                sh '''
                    mvn test \
                    -Dsurefire.suiteXmlFiles=src/test/resources/smoke.xml \
                    -Dbrowser=${BROWSER} \
                    -Dbase.url=${BASE_URL} \
                    -Dheadless=true
                '''
            }
        }

        stage('Run Regression Tests') {
            when {
                branch 'main'
            }
            steps {
                sh '''
                    mvn test \
                    -Dsurefire.suiteXmlFiles=src/test/resources/regression.xml \
                    -Dbrowser=${BROWSER} \
                    -Dheadless=true
```

```
                '''
            }
        }
    }

    post {
        always {
            publishHTML(target: [
                allowMissing: true,
                alwaysLinkToLastBuild: true,
                keepAll: true,
                reportDir: 'reports',
                reportFiles: 'report.html',
                reportName: 'Extent Test Report'
            ])
            junit 'target/surefire-reports/*.xml'
        }
        failure {
            emailext(
                subject: "FAILED: ${JOB_NAME} #${BUILD_NUMBER}",
                body: "Tests failed. See: ${BUILD_URL}",
                to: 'team@example.com'
            )
        }
        success {
            echo 'All tests passed!'
        }
    }
}
```

7.4 Maven Surefire Plugin Configuration

The Maven Surefire Plugin is responsible for running TestNG tests during the Maven build lifecycle:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
      <configuration>
        <suiteXmlFiles>
          <suiteXmlFile>
            ${suiteFile}
          </suiteXmlFile>
```

```
        </suiteXmlFiles>
        <argLine>-Xmx1024m</argLine>
        <testFailureIgnore>false</testFailureIgnore>
        <systemPropertyVariables>
          <browser>${browser}</browser>
          <base.url>${baseUrl}</base.url>
          <headless>${headless}</headless>
        </systemPropertyVariables>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Running Tests from Maven Command Line

```
# Run all tests
mvn clean test

# Run with custom suite file
mvn clean test -DsuiteFile=src/test/resources/smoke.xml

# Run with environment overrides
mvn clean test -Dbrowser=firefox -Dbase.url=https://staging.example.com

# Run in headless mode
mvn clean test -Dheadless=true
```

7.5 Jenkins Job Configuration Best Practices

Build Triggers

- Poll SCM: Check Git for changes on a schedule (e.g., every 5 minutes): H/5 * * * *
- GitHub Webhook: Trigger immediately on every push — recommended for fast feedback
- Scheduled Builds: Run full regression nightly: 0 0 * * * (midnight daily)
- Upstream Trigger: Run after a dependent job completes successfully

Workspace Cleanup

```
// In Jenkinsfile — clean before build
stage('Cleanup') {
    steps {
        cleanWs() // Clean Jenkins workspace
        sh 'rm -rf screenshots/* reports/* logs/*'
    }
}
```

> ✓ **PRO TIP**
> Always run tests in headless mode in Jenkins (Headless recommended but not mandatory). Install Xvfb (virtual display) as an alternative if some tests require a visible browser. Never use headed browsers directly in Jenkins.

7.6 Parallel Test Execution in Jenkins

```
pipeline {
    agent any
    stages {
        stage('Parallel Tests') {
            parallel {
                stage('Chrome Tests') {
                    steps {
                        sh 'mvn test -Dbrowser=chrome -Dheadless=true'
                    }
                }
                stage('Firefox Tests') {
                    steps {
                        sh 'mvn test -Dbrowser=firefox -Dheadless=true'
                    }
                }
                stage('Edge Tests') {
                    steps {
                        sh 'mvn test -Dbrowser=edge -Dheadless=true'
                    }
                }
            }
        }
    }
}
```

Chapter 8: Real-World Projects & Interview Preparation

8.1 Project 1 — E-Commerce Automation Framework

This end-to-end project demonstrates a complete automation framework for an e-commerce application covering login, product search, cart operations, and checkout.

Project Features

- Page Object Model with BasePage and BaseTest
- Data-driven testing with Excel for user credentials and product data
- Parallel execution across Chrome and Firefox
- ExtentReports with screenshot on failure
- Log4j2 integration for structured logging
- Jenkins pipeline with smoke and regression stages
- ConfigReader for multi-environment support

Test Coverage

Test Module	Test Cases	Group
Login	Valid login, Invalid password, Empty fields, Forgot password	smoke, regression
Product Search	Search by keyword, Filter by category, Sort results	regression
Product Detail	View product, Check stock, Check price, Image gallery	regression
Shopping Cart	Add item, Remove item, Update quantity, Empty cart	smoke, regression
Checkout	Guest checkout, Registered checkout, Payment validation	regression
User Account	Profile update, Order history, Address management	regression

8.2 Project 2 — Banking Application Framework

A more complex project covering secure login, fund transfers, account management, and transaction history with API validation using REST Assured alongside Selenium.

Key Automation Challenges Addressed

- Handling session timeouts and re-authentication

- Validating transaction records in data tables
- File download verification (statement PDF)
- Cross-browser testing for compliance
- Sensitive data masking in logs and reports

8.3 Common Automation Challenges & Solutions

Challenge	Solution
Flaky Tests	Use explicit waits, retry logic via TestNG @Test(retryAnalyzer), stable locators
Stale Element Exception	Re-locate element in a retry block, avoid storing stale references
Dynamic Content	Wait for dynamic element with FluentWait, use contains() in XPath
Authentication Popups	Use ChromeOptions to pass credentials, or handle via Selenium switchTo().alert()
File Upload	Use sendKeys() with file path on input[type=file] elements
Shadow DOM	Use JS executeScript to query shadowRoot then find elements
Captcha	Use test environments where CAPTCHA is disabled; never automate CAPTCHA breaking

8.4 Top Interview Questions & Answers

Selenium Questions

Q: What is the difference between findElement() and findElements()?

```
// findElement() - returns single WebElement, throws NoSuchElementException if not found
WebElement btn = driver.findElement(By.id("submit"));


// findElements() - returns List<WebElement>, returns empty list if not found
List<WebElement> items = driver.findElements(By.className("item"));
```

Q: How do you handle dynamic elements in Selenium?

Use partial attribute matching in locators. For example, if an element's id changes dynamically like 'btn_12345', use: By.xpath("//button[contains(@id,'btn_')]") or By.cssSelector("button[id^='btn_']"). For content that loads asynchronously, use explicit waits with ExpectedConditions.

Q: Explain the difference between implicit wait, explicit wait and fluent wait.

- Implicit Wait: Global setting applied to all findElement() calls. The driver polls for the element until it appears or timeout expires.
- Explicit Wait: Targeted wait for a specific condition (visibility, clickability, presence) on a specific element. More precise and preferred.

- Fluent Wait: An explicit wait with custom polling interval and ability to ignore specific exceptions during the wait period.

TestNG Questions

Q: What is the difference between @BeforeMethod and @BeforeClass?

@BeforeMethod runs before every @Test method in the class — use it to set up preconditions for each test (e.g., launch browser, navigate to URL). @BeforeClass runs once before the first test method in the class — use it for expensive setup that can be shared across tests (e.g., establishing database connection).

Q: How do you retry failed tests in TestNG?

```
public class RetryAnalyzer implements IRetryAnalyzer {
    private int retryCount = 0;
    private static final int MAX_RETRY = 2;

    @Override
    public boolean retry(ITestResult result) {
        if (retryCount < MAX_RETRY) {
            retryCount++;
            return true; // Retry
        }
        return false; // Stop retrying
    }
}

@Test(retryAnalyzer = RetryAnalyzer.class)
public void flakyTest() { ... }
```

Framework & Architecture Questions

Q: What is Page Object Model and why is it important?

POM is a design pattern that creates an object repository of web pages. Each page is represented as a class with locators as private fields and user actions as public methods. Tests interact with pages only through these methods. POM improves maintainability (locator changes only need updating in one place), readability (tests read like user stories), and reusability (page methods shared across multiple tests).

Q: How do you implement cross-browser testing?

```
public class DriverFactory {
    public static WebDriver createDriver(String browser) {
        return switch (browser.toLowerCase()) {
            case "chrome" -> {
                WebDriverManager.chromedriver().setup();
                yield new ChromeDriver(getChromeOptions());
```

```
            }
            case "firefox" -> {
                WebDriverManager.firefoxdriver().setup();
                yield new FirefoxDriver(getFirefoxOptions());
            }
            case "edge" -> {
                WebDriverManager.edgedriver().setup();
                yield new EdgeDriver(getEdgeOptions());
            }
            default -> throw new IllegalArgumentException(
                "Unsupported browser: " + browser);
        };
    }
}
```

8.5 Career Roadmap for Automation Engineers

Junior Automation Engineer (0-2 years)

- Core Java, OOP concepts
- Selenium WebDriver basics
- TestNG/JUnit fundamentals
- Basic POM implementation
- SQL basics for test data validation

Mid-Level Automation Engineer (2-5 years)

- Advanced framework design (POM, hybrid, keyword-driven)
- CI/CD with Jenkins, GitHub Actions
- API testing with REST Assured
- BDD with Cucumber/Gherkin
- Docker for Selenium Grid

Senior/Lead Automation Engineer (5+ years)

- Framework architecture design from scratch
- Selenium Grid and cloud testing (BrowserStack, Sauce Labs)
- Performance testing integration (JMeter)
- Test strategy and test planning for teams
- Mentoring junior engineers and code reviews

PRO TIP

Build a portfolio on GitHub with 2-3 complete automation framework projects demonstrating POM, data-driven testing, parallel execution, and Jenkins integration. Recruiters actively look for practical GitHub portfolios.

Chapter 9: Selenium 4 — Advanced Features & New APIs

Selenium 4 introduced a major overhaul including full W3C WebDriver compliance, Chrome DevTools Protocol (CDP) integration, relative locators, improved window/tab management, and a completely revamped documentation. This chapter covers every significant Selenium 4 feature with practical examples.

9.1 Chrome DevTools Protocol (CDP) Integration

Selenium 4 provides built-in access to Chrome DevTools Protocol, enabling capabilities that were previously impossible with WebDriver alone — such as network interception, geolocation mocking, console log capture, and performance metrics.

9.1.1 Network Interception & Mocking

```
import org.openqa.selenium.devtools.DevTools;

// NOTE: CDP version depends on the installed Chrome browser version.

import org.openqa.selenium.devtools.v120.network.Network;
import org.openqa.selenium.devtools.v120.network.model.*;

ChromeDriver driver = new ChromeDriver();
DevTools devTools = driver.getDevTools();
devTools.createSession();

// Enable network monitoring
devTools.send(Network.enable(Optional.empty(),
    Optional.empty(), Optional.empty()));

// Capture all network requests
devTools.addListener(Network.requestWillBeSent(), request -> {
    System.out.println("URL: " + request.getRequest().getUrl());
    System.out.println("Method: " + request.getRequest().getMethod());
});

// Capture all network responses
devTools.addListener(Network.responseReceived(), response -> {
    System.out.println("Status: " + response.getResponse().getStatus());
    System.out.println("URL: " + response.getResponse().getUrl());
});

driver.get("https://example.com");
```

9.1.2 Blocking Network Requests

```
// Block specific URLs (e.g., ads, analytics)
devTools.send(Network.enable(Optional.empty(),
    Optional.empty(), Optional.empty()));
devTools.send(Network.setBlockedURLs(
    List.of("*google-analytics*", "*ads*", "*tracking*")));

driver.get("https://example.com"); // Loads without ads/analytics
```

9.1.3 Mocking Geolocation

```
import org.openqa.selenium.devtools.v120.emulation.Emulation;

// Set fake geolocation (Mumbai, India)
devTools.send(Emulation.setGeolocationOverride(
    Optional.of(19.0760),  // latitude
    Optional.of(72.8777),  // longitude
    Optional.of(1)         // accuracy
));

driver.get("https://maps.google.com");
// Google Maps will show Mumbai as current location
```

9.1.4 Capture Console Logs via CDP

```
import org.openqa.selenium.devtools.v120.log.Log;

devTools.send(Log.enable());
devTools.addListener(Log.entryAdded(), logEntry -> {
    System.out.println("[" + logEntry.getLevel() + "] "
        + logEntry.getText());
});

driver.get("https://example.com");
// All browser console logs are now captured in Java
```

9.1.5 Simulate Network Conditions

```
// Simulate slow 3G network
devTools.send(Network.emulateNetworkConditions(
    false,        // offline
    100,          // latency in ms
```

```
    50000,        // downloadThroughput (bytes/sec ~ 50KB/s)
    20000,        // uploadThroughput
    Optional.of(ConnectionType.CELLULAR3G)
));
```

> ✓ **PRO TIP**
> Use network throttling in CI to simulate real-world conditions. Tests passing on fast CI networks may fail on slow mobile networks used by real users.

9.2 Relative Locators

Selenium 4 introduces relative locators (also called friendly locators), allowing you to find elements based on their visual position relative to other elements — mimicking how humans describe UI elements.

```
import static org.openqa.selenium.support.locators.RelativeLocator.*;

// Find the password field BELOW the username field
WebElement password = driver.findElement(
    with(By.tagName("input")).below(By.id("username")));

// Find label to the LEFT of a checkbox
WebElement label = driver.findElement(
    with(By.tagName("label")).toLeftOf(By.id("acceptTerms")));

// Find button ABOVE the footer
WebElement submitBtn = driver.findElement(
    with(By.tagName("button")).above(By.id("footer")));

// Find element NEAR another (within 50px)
WebElement hint = driver.findElement(
    with(By.className("help-text")).near(By.id("email")));

// Combine: find input to the RIGHT of a label
WebElement input = driver.findElement(
    with(By.tagName("input")).toRightOf(By.id("emailLabel")));
```

> ℹ **NOTE**
> Relative locators use JavaScript's getBoundingClientRect() under the hood. They work best for stable UI layouts. Avoid using them for highly dynamic or responsive layouts where element positions shift.

9.3 New Window & Tab Management

Selenium 4 provides clean APIs to open new browser windows or tabs programmatically:

```
// Open a new tab and switch to it
driver.switchTo().newWindow(WindowType.TAB);
driver.get("https://new-tab-url.com");

// Open a new browser window
driver.switchTo().newWindow(WindowType.WINDOW);
driver.get("https://new-window-url.com");

// Get all window handles
Set<String> handles = driver.getWindowHandles();

// Switch back to original tab (first handle)
String originalTab = driver.getWindowHandle();
driver.switchTo().newWindow(WindowType.TAB);
// ... do work in new tab ...
driver.close(); // Close new tab
driver.switchTo().window(originalTab); // Return to original
```

9.4 Screenshot Improvements — Element Screenshots

Selenium 4 allows capturing screenshots of individual WebElements, not just the full page:

```
// Full page screenshot (as before)
File pageSS = ((TakesScreenshot) driver)
    .getScreenshotAs(OutputType.FILE);

// Element-level screenshot (NEW in Selenium 4)
WebElement loginForm = driver.findElement(By.id("login-form"));
File elementSS = loginForm.getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(elementSS, new File("./login-form.png"));

// Get screenshot as Base64 for embedding in reports
String base64SS = loginForm.getScreenshotAs(OutputType.BASE64);
// Embed in ExtentReports:
extentTest.addScreenCaptureFromBase64String(base64SS,
    "Login Form Screenshot");
```

9.5 WebDriver BiDi (Bidirectional) Protocol

Selenium 4.6+ introduces WebDriver BiDi (bidirectional communication), which enables real-time event listening directly from the browser without polling. This is the next generation after CDP.

```
// Register JS exception listener (BiDi)
((HasLogEvents) driver).onLogEvent(
```

```
    ConsoleLogEntry.class,
    entry -> System.out.println("Console: " + entry.getText())
);

((HasLogEvents) driver).onLogEvent(
    JavascriptLogEntry.class,
    entry -> System.out.println("JS Error: " + entry.getText())
);
```

9.6 Selenium Grid 4

Selenium Grid 4 has been completely rewritten with a new architecture that supports standalone, hub-node, and fully distributed modes with Docker and Kubernetes integration.

Grid 4 Architecture Modes

Mode	Use Case	How to Start
Standalone	Local single machine testing	java -jar selenium-server.jar standalone
Hub + Node	Traditional hub-node distributed	java -jar selenium-server.jar hub / node
Distributed	Enterprise large-scale grid	Separate Router, Distributor, Node services
Docker	Containerized parallel execution	docker-compose with selenium/node-chrome

Docker Compose for Selenium Grid

```
version: '3'
services:
  selenium-hub:
    image: selenium/hub:latest
    ports:
      - "4442:4442"
      - "4443:4443"
      - "4444:4444"

  chrome-node:
    image: selenium/node-chrome:latest
    shm_size: '2gb'
    depends_on:
      - selenium-hub
    environment:
      - SE_EVENT_BUS_HOST=selenium-hub
      - SE_EVENT_BUS_PUBLISH_PORT=4442
```

```
      - SE_EVENT_BUS_SUBSCRIBE_PORT=4443
    scale: 3  # 3 Chrome nodes

  firefox-node:
    image: selenium/node-firefox:latest
    shm_size: '2gb'
    depends_on:
      - selenium-hub
    environment:
      - SE_EVENT_BUS_HOST=selenium-hub
      - SE_EVENT_BUS_PUBLISH_PORT=4442
      - SE_EVENT_BUS_SUBSCRIBE_PORT=4443
```

Connecting Tests to Grid

```
// Point tests to the Selenium Grid Hub
ChromeOptions options = new ChromeOptions();
options.addArguments("--headless=new");

WebDriver driver = new RemoteWebDriver(
    new URL("http://localhost:4444"),
    options
);
```

PRO TIP

Use Selenium Grid with Docker for consistent, isolated, scalable parallel test execution. Each container provides a fresh browser instance, eliminating state pollution between tests.

Chapter 10: BDD with Cucumber & Gherkin

Behavior-Driven Development (BDD) bridges the communication gap between business stakeholders, QA engineers, and developers. Cucumber is the most popular BDD framework for Java, using Gherkin — a plain English syntax — to describe test scenarios. When combined with Selenium and TestNG/JUnit, Cucumber creates living documentation that serves as both specification and automated test.

10.1 BDD Concepts & Gherkin Syntax

Gherkin uses a structured English-like language to describe application behavior in terms of Features, Scenarios, and Steps. Every line in a Gherkin file maps to a Java method called a Step Definition.

Core Gherkin Keywords

Keyword	Purpose	Example
Feature	High-level description of the feature	Feature: User Authentication
Scenario	A single test case / example	Scenario: Successful login with valid credentials
Scenario Outline	Parameterized scenario with Examples table	Scenario Outline: Login with <role>
Given	Precondition / initial context	Given the user is on the login page
When	Action performed by the user	When the user enters valid credentials
Then	Expected outcome / assertion	Then the dashboard should be displayed
And / But	Additional Given/When/Then steps	And the welcome message shows the username
Background	Steps shared by all scenarios in a feature	Background: Given the user navigates to login
Examples	Data table for Scenario Outline	| username | password | expected |
@Tag	Categorize scenarios for selective runs	@smoke, @regression, @login

10.2 Maven Dependencies for Cucumber

```
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>7.15.0</version>
  <scope>test</scope>
```

```
</dependency>
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-testng</artifactId>
  <version>7.15.0</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-picocontainer</artifactId>
  <version>7.15.0</version>
  <scope>test</scope>
</dependency>
```

10.3 Feature File Example — Login

```
# src/test/resources/features/login.feature

@login @smoke
Feature: User Login
  As a registered user
  I want to log into the application
  So that I can access my dashboard

  Background:
    Given the user navigates to the login page

  @positive
  Scenario: Successful login with valid credentials
    When the user enters username "admin" and password "Admin@123"
    And the user clicks the Login button
    Then the user should be redirected to the Dashboard
    And the welcome message should display "Welcome, Admin"

  @negative
  Scenario: Login fails with incorrect password
    When the user enters username "admin" and password "wrongpass"
    And the user clicks the Login button
    Then an error message "Invalid credentials" should be displayed

  @data-driven
  Scenario Outline: Login with multiple user roles
    When the user enters username "<username>" and password "<password>"
    And the user clicks the Login button
```

```
    Then the user should see the "<expectedPage>" page

    Examples:
      | username  | password   | expectedPage |
      | admin     | Admin@123  | Dashboard    |
      | manager   | Mgr@456    | Reports      |
      | viewer    | View@789   | ReadOnly     |
```

10.4 Step Definitions

```
import io.cucumber.java.en.*;
import org.testng.Assert;

public class LoginSteps {
    private WebDriver driver;
    private LoginPage loginPage;
    private DashboardPage dashboardPage;

    // Constructor injection via PicoContainer
    public LoginSteps(SharedDriver sharedDriver) {
        this.driver = sharedDriver.getDriver();
    }

    @Given("the user navigates to the login page")
    public void navigateToLoginPage() {
        driver.get(ConfigReader.get("base.url") + "/login");
        loginPage = new LoginPage(driver);
    }

    @When("the user enters username {string} and password {string}")
    public void enterCredentials(String username, String password) {
        loginPage.enterUsername(username);
        loginPage.enterPassword(password);
    }

    @When("the user clicks the Login button")
    public void clickLogin() {
        loginPage.clickLoginButton();
    }

    @Then("the user should be redirected to the Dashboard")
    public void verifyDashboardPage() {
        dashboardPage = new DashboardPage(driver);
```

```
        Assert.assertTrue(dashboardPage.isLoaded(),
            "Dashboard page did not load");
    }

    @Then("the welcome message should display {string}")
    public void verifyWelcomeMessage(String expectedMsg) {
        Assert.assertEquals(dashboardPage.getWelcomeMessage(),
            expectedMsg);
    }

    @Then("an error message {string} should be displayed")
    public void verifyErrorMessage(String expectedError) {
        Assert.assertEquals(loginPage.getErrorMessage(),
            expectedError);
    }
}
```

10.5 TestNG Runner for Cucumber

```
@CucumberOptions(
    features = "src/test/resources/features",
    glue = {"stepDefinitions", "hooks"},
    tags = "@smoke",
    plugin = {
        "pretty",
        "html:target/cucumber-reports/report.html",
        "json:target/cucumber-reports/report.json",
        "io.qameta.allure.cucumber7jvm.AllureCucumber7Jvm"
    },
    monochrome = true,
    dryRun = false
)
public class TestRunner extends AbstractTestNGCucumberTests {

    @Override
    @DataProvider(parallel = true) // Run scenarios in parallel
    public Object[][] scenarios() {
        return super.scenarios();
    }
}
```

10.6 Cucumber Hooks

```
import io.cucumber.java.*;

public class Hooks {
    private final SharedDriver sharedDriver;

    public Hooks(SharedDriver sharedDriver) {
        this.sharedDriver = sharedDriver;
    }

    @Before(order = 1)
    public void beforeScenario(Scenario scenario) {
        System.out.println("Starting: " + scenario.getName());
        sharedDriver.initDriver();
    }

    @After(order = 1)
    public void afterScenario(Scenario scenario) {
        if (scenario.isFailed()) {
            // Capture screenshot on failure
            byte[] screenshot = ((TakesScreenshot) sharedDriver.getDriver())
                .getScreenshotAs(OutputType.BYTES);
            scenario.attach(screenshot, "image/png", "Failure Screenshot");
        }
        sharedDriver.quitDriver();
    }

    @BeforeStep
    public void beforeStep(Scenario scenario) {
        // Runs before every step
    }

    @AfterStep
    public void afterStep(Scenario scenario) {
        // Capture step screenshot for reporting
        byte[] ss = ((TakesScreenshot) sharedDriver.getDriver())
            .getScreenshotAs(OutputType.BYTES);
        scenario.attach(ss, "image/png", "Step: " + scenario.getName());
    }
}
```

10.7 Cucumber Project Structure

```
src/
├── test/
```

```
│   ├── java/
│   │   ├── runner/
│   │   │   └── TestRunner.java          # CucumberOptions + TestNG
│   │   ├── stepDefinitions/
│   │   │   ├── LoginSteps.java
│   │   │   ├── CartSteps.java
│   │   │   └── CommonSteps.java
│   │   ├── hooks/
│   │   │   └── Hooks.java               # Before/After scenario
│   │   └── context/
│   │       └── SharedDriver.java        # Thread-safe WebDriver
│   └── resources/
│       └── features/
│           ├── login.feature
│           ├── cart.feature
│           └── checkout.feature
```

PRO TIP

Keep step definitions small and focused. Each step definition class should correspond to a single feature area (e.g., LoginSteps, CartSteps). Avoid creating a single giant StepDefinitions class — it becomes unmaintainable quickly.

Chapter 11: API Testing with REST Assured

Modern automation frameworks are incomplete without API testing. REST Assured is the industry-standard Java library for testing RESTful APIs. It provides a fluent, BDD-style DSL that makes API tests highly readable. Combining Selenium UI tests with REST Assured API tests creates a comprehensive test strategy.

11.1 Why API Testing Matters

- APIs are the backbone of modern web applications — UI tests only cover the surface
- API tests run 10-100x faster than equivalent UI tests
- API tests are more stable — UI changes don't break them
- API tests can validate business logic independent of UI
- API tests can be used to set up/tear down test data for UI tests

> **ℹ HYBRID STRATEGY**
> A best practice is to use REST Assured APIs for test data setup (creating users, orders, etc.) before running Selenium UI tests. This is faster and more reliable than navigating through the UI to create preconditions.

11.2 Maven Setup

```
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>rest-assured</artifactId>
  <version>5.4.0</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>json-path</artifactId>
  <version>5.4.0</version>
</dependency>
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>json-schema-validator</artifactId>
  <version>5.4.0</version>
</dependency>
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.17.1</version>
```

```
</dependency>
```

11.3 REST Assured Basics — Given/When/Then

REST Assured follows the same Given-When-Then BDD structure, making tests very readable:

```
import static io.restassured.RestAssured.*;
import static org.hamcrest.Matchers.*;

// GET request with validation
given()
    .baseUri("https://api.example.com")
    .header("Accept", "application/json")
    .queryParam("page", 1)
    .queryParam("size", 10)
.when()
    .get("/users")
.then()
    .statusCode(200)
    .contentType(ContentType.JSON)
    .body("data.size()", greaterThan(0))
    .body("data[0].email", notNullValue())
    .body("totalCount", greaterThanOrEqualTo(1));
```

11.4 CRUD Operations

POST — Create Resource

```
// Using HashMap body
Map<String, Object> requestBody = new HashMap<>();
requestBody.put("name", "John Doe");
requestBody.put("email", "john@example.com");
requestBody.put("role", "admin");

Response response =
given()
    .baseUri("https://api.example.com")
    .header("Authorization", "Bearer " + authToken)
    .contentType(ContentType.JSON)
    .body(requestBody)
.when()
    .post("/users")
.then()
    .statusCode(201)
    .body("id", notNullValue())
```

```
    .body("name", equalTo("John Doe"))
    .extract().response();

// Extract created user's ID for later use
String userId = response.jsonPath().getString("id");
```

PUT — Update Resource

```
given()
    .baseUri("https://api.example.com")
    .header("Authorization", "Bearer " + authToken)
    .contentType(ContentType.JSON)
    .body("{\"name\": \"Jane Doe\", \"role\": \"viewer\"}")
.when()
    .put("/users/" + userId)
.then()
    .statusCode(200)
    .body("name", equalTo("Jane Doe"));
```

DELETE — Remove Resource

```
given()
    .baseUri("https://api.example.com")
    .header("Authorization", "Bearer " + authToken)
.when()
    .delete("/users/" + userId)
.then()
    .statusCode(204); // 204 No Content = successful deletion
```

11.5 Authentication Patterns

Bearer Token (JWT)

```
String token = getAuthToken(); // Your auth helper method

given()
    .auth().oauth2(token)   // OR .header("Authorization", "Bearer " + token)
    .get("/protected-endpoint")
.then()
    .statusCode(200);
```

Basic Authentication

```
given()
```

```
    .auth().basic("username", "password")
    .get("/api/data")
.then()
    .statusCode(200);
```

Obtaining JWT Token (Login Flow)

```
public static String getAuthToken() {
    Map<String, String> credentials = new HashMap<>();
    credentials.put("username", "admin");
    credentials.put("password", "Admin@123");

    return given()
        .baseUri("https://api.example.com")
        .contentType(ContentType.JSON)
        .body(credentials)
    .when()
        .post("/auth/login")
    .then()
        .statusCode(200)
        .extract()
        .path("token");
}
```

11.6 Request & Response Specification

RequestSpecification and ResponseSpecification allow you to define reusable request/response configurations, eliminating repetition across test classes:

```
public class ApiConfig {
    public static RequestSpecification requestSpec() {
        return new RequestSpecBuilder()
            .setBaseUri(ConfigReader.get("api.base.url"))
            .setContentType(ContentType.JSON)
            .addHeader("Accept", "application/json")
            .addHeader("Authorization",
                "Bearer " + getAuthToken())
            .log(LogDetail.ALL)
            .build();
    }

    public static ResponseSpecification responseSpec() {
        return new ResponseSpecBuilder()
            .expectStatusCode(200)
            .expectContentType(ContentType.JSON)
```

```
            .expectResponseTime(lessThan(3000L))
            .build();
    }
}

// Usage — much cleaner tests
given()
    .spec(ApiConfig.requestSpec())
.when()
    .get("/users")
.then()
    .spec(ApiConfig.responseSpec())
    .body("users", hasSize(greaterThan(0)));
```

11.7 JSON Schema Validation

Validating API responses against a JSON schema ensures the response structure matches the contract, even if values change:

```
// user-schema.json (place in src/test/resources/schemas/)
{
  "$schema": "http://json-schema.org/draft-07/schema",
  "type": "object",
  "required": ["id", "name", "email"],
  "properties": {
    "id": { "type": "integer" },
    "name": { "type": "string", "minLength": 1 },
    "email": { "type": "string", "format": "email" },
    "role": { "type": "string",
              "enum": ["admin", "manager", "viewer"] }
  }
}

// In test:
given()
    .spec(ApiConfig.requestSpec())
.when()
    .get("/users/1")
.then()
    .statusCode(200)
    .body(matchesJsonSchemaInClasspath(
        "schemas/user-schema.json"));
```

11.8 Using POJOs for Request/Response

Using Java POJOs with Jackson makes API tests type-safe and easier to maintain:

```
// POJO for User
public class User {
    private int id;
    private String name;
    private String email;
    private String role;
    // Getters, Setters, @JsonIgnoreProperties(ignoreUnknown=true)
}

// Deserialize response to POJO
User user = given()
    .spec(ApiConfig.requestSpec())
.when()
    .get("/users/1")
.then()
    .statusCode(200)
    .extract()
    .as(User.class);

Assert.assertEquals(user.getName(), "John Doe");
Assert.assertEquals(user.getRole(), "admin");

// Deserialize list
List<User> users = given()
    .spec(ApiConfig.requestSpec())
.when()
    .get("/users")
.then()
    .extract()
    .jsonPath()
    .getList("data", User.class);
```

PRO TIP

Always use POJOs for complex API requests and responses. String-based JSON manipulation is error-prone and hard to maintain. Jackson with @JsonIgnoreProperties(ignoreUnknown = true) handles forward compatibility gracefully.

11.9 Combining API & UI Tests

A powerful pattern is to use REST Assured for test data setup and teardown, and Selenium only for UI interaction:

```
public class CheckoutTest extends BaseTest {


    private String testUserId;
```

```
    private String testOrderId;

    @BeforeMethod
    public void setupTestData() {
        // Use API to create test user (faster than UI)
        testUserId = ApiUtils.createUser("testuser@example.com");
        // Use API to add items to cart
        testOrderId = ApiUtils.createCartWithItems(
            testUserId, List.of("SKU001", "SKU002"));
    }

    @Test
    public void testCheckoutFlow() {
        // Now use Selenium only for UI checkout steps
        driver.get(ConfigReader.get("base.url") + "/cart");
        CartPage cartPage = new CartPage(driver);
        Assert.assertEquals(cartPage.getItemCount(), 2);
        // ... continue UI test
    }

    @AfterMethod
    public void cleanupTestData() {
        // Use API to clean up (no browser needed)
        ApiUtils.deleteUser(testUserId);
    }
}
```

Chapter 12: Framework Best Practices & Design Patterns

Building a maintainable, scalable test automation framework requires applying proven design patterns and engineering best practices. This chapter covers the most impactful patterns and practices used in enterprise-grade automation frameworks.

12.1 Design Patterns in Test Automation

12.1.1 Singleton Pattern — WebDriver Management

Ensure only one WebDriver instance exists per thread:

```
public class DriverManager {
    private static final ThreadLocal<WebDriver> tlDriver
        = new ThreadLocal<>();

    private DriverManager() {}  // Prevent instantiation

    public static WebDriver getDriver() {
        if (tlDriver.get() == null) {
            throw new IllegalStateException(
                "Driver not initialized. Call setDriver() first.");
        }
        return tlDriver.get();
    }

    public static void setDriver(WebDriver driver) {
        tlDriver.set(driver);
    }

    public static void quitDriver() {
        if (tlDriver.get() != null) {
            tlDriver.get().quit();
            tlDriver.remove(); // CRITICAL: Prevent memory leaks
        }
    }
}
```

12.1.2 Factory Pattern — Browser Creation

```
public class DriverFactory {
    public static WebDriver createDriver(String browserName) {
        BrowserType browser = BrowserType.fromString(browserName);
```

```
        return switch (browser) {
            case CHROME -> createChromeDriver();
            case FIREFOX -> createFirefoxDriver();
            case EDGE -> createEdgeDriver();
            case SAFARI -> createSafariDriver();
        };
    }

    private static WebDriver createChromeDriver() {
        WebDriverManager.chromedriver().setup();
        ChromeOptions opts = new ChromeOptions();
        if (ConfigReader.getBoolean("headless")) {
            opts.addArguments("--headless=new",
                "--no-sandbox", "--disable-dev-shm-usage");
        }
        opts.addArguments("--start-maximized");
        opts.addArguments("--disable-notifications");
        return new ChromeDriver(opts);
    }

    private static WebDriver createFirefoxDriver() {
        WebDriverManager.firefoxdriver().setup();
        FirefoxOptions opts = new FirefoxOptions();
        if (ConfigReader.getBoolean("headless")) {
            opts.addArguments("--headless=new");
        }
        return new FirefoxDriver(opts);
    }
}
```

12.1.3 Builder Pattern — Test Data Creation

The Builder pattern creates clean, readable test data objects:

```
public class User {
    private final String firstName;
    private final String lastName;
    private final String email;
    private final String password;
    private final String role;
    private final String phone;

    private User(Builder builder) {
        this.firstName = builder.firstName;
        this.lastName = builder.lastName;
```

```
        this.email = builder.email;
        this.password = builder.password;
        this.role = builder.role;
        this.phone = builder.phone;
    }

    public static class Builder {
        private String firstName = "Test";
        private String lastName = "User";
        private String email = "test" + System.currentTimeMillis()
                               + "@example.com";
        private String password = "Test@1234";
        private String role = "viewer";
        private String phone = "";

        public Builder firstName(String val) { firstName=val; return this; }
        public Builder lastName(String val) { lastName=val; return this; }
        public Builder email(String val) { email=val; return this; }
        public Builder role(String val) { role=val; return this; }
        public User build() { return new User(this); }
    }
}

// Usage — very readable
User adminUser = new User.Builder()
    .firstName("John")
    .lastName("Doe")
    .email("john.doe@test.com")
    .role("admin")
    .build();
```

12.1.4 Strategy Pattern — Wait Strategies

```
public interface WaitStrategy {
    WebElement waitFor(WebDriver driver, By locator);
}

public class VisibilityWait implements WaitStrategy {
    @Override
    public WebElement waitFor(WebDriver driver, By locator) {
        return new WebDriverWait(driver, Duration.ofSeconds(15))
            .until(ExpectedConditions.visibilityOfElementLocated(locator));
    }
}
```

```
public class ClickableWait implements WaitStrategy {
    @Override
    public WebElement waitFor(WebDriver driver, By locator) {
        return new WebDriverWait(driver, Duration.ofSeconds(15))
            .until(ExpectedConditions.elementToBeClickable(locator));
    }
}

// Usage in BasePage
public WebElement findElement(By locator, WaitStrategy strategy) {
    return strategy.waitFor(driver, locator);
}
element = findElement(By.id("btn"), new ClickableWait());
```

12.2 Retry Mechanism for Flaky Tests

```
public class RetryAnalyzer implements IRetryAnalyzer {
    private int retryCount = 0;
    private static final int MAX_RETRY_COUNT
        = Integer.parseInt(ConfigReader.get("max.retry"));

    @Override
    public boolean retry(ITestResult result) {
        if (!result.isSuccess() && retryCount < MAX_RETRY_COUNT) {
            retryCount++;
            System.out.println("Retrying " + result.getName()
                + " | Attempt " + retryCount),
            return true;
        }
        return false;
    }
}

// Apply to all tests automatically via listener
public class RetryListener implements IAnnotationTransformer {
    @Override
    public void transform(ITestAnnotation annotation,
            Class testClass, Constructor testConstructor,
            Method testMethod) {
        annotation.setRetryAnalyzer(RetryAnalyzer.class);
    }
}
```

12.3 Custom Annotation for Test Metadata

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface TestInfo {
    String testId();
    String description();
    String[] tags() default {};
    String author() default "QA Team";
    String priority() default "Medium";
}

// Usage on test methods
@Test
@TestInfo(
    testId = "TC-LOGIN-001",
    description = "Verify successful login with admin credentials",
    tags = {"smoke", "regression"},
    author = "John Doe",
    priority = "High"
)
public void testAdminLogin() { ... }

// Read annotation in listener
TestInfo info = result.getMethod()
    .getConstructorOrMethod().getMethod()
    .getAnnotation(TestInfo.class);
if (info != null) {
    extentTest.info("Test ID: " + info.testId());
}
```

12.4 Environment Management

Professional frameworks support multiple environments without code changes:

```
# src/test/resources/environments/dev.properties
base.url=https://dev.example.com
api.base.url=https://api.dev.example.com
db.url=jdbc:mysql://dev-db:3306/testdb

# src/test/resources/environments/staging.properties
base.url=https://staging.example.com
api.base.url=https://api.staging.example.com

# src/test/resources/environments/prod.properties
```

```
base.url=https://example.com
api.base.url=https://api.example.com

// EnvironmentManager.java
public class EnvironmentManager {
    private static Properties props;

    static {
        String env = System.getProperty("env", "staging");
        String path = "src/test/resources/environments/"
                    + env + ".properties";
        props = new Properties();
        try (FileInputStream fis = new FileInputStream(path)) {
            props.load(fis);
        } catch (IOException e) {
            throw new RuntimeException("Env file not found: " + path);
        }
    }

    public static String get(String key) {
        return props.getProperty(key);
    }
}

// Run on staging:  mvn test -Denv=staging
// Run on dev:      mvn test -Denv=dev
// Run on prod:     mvn test -Denv=prod
```

12.5 Database Validation in Tests

For thorough test validation, especially for backend-heavy operations, verifying database state directly provides an extra layer of confidence:

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
</dependency>

public class DatabaseUtils {
    private static Connection connection;

    public static Connection getConnection() {
        if (connection == null) {
```

```
            try {
                connection = DriverManager.getConnection(
                    EnvironmentManager.get("db.url"),
                    EnvironmentManager.get("db.user"),
                    EnvironmentManager.get("db.pass")
                );
            } catch (SQLException e) {
                throw new RuntimeException("DB connection failed", e);
            }
        }
        return connection;
    }

    public static String getStringValue(String query) {
        try (Statement stmt = getConnection().createStatement();
             ResultSet rs = stmt.executeQuery(query)) {
            return rs.next() ? rs.getString(1) : null;
        } catch (SQLException e) {
            throw new RuntimeException("Query failed: " + query, e);
        }
    }
}

// In test — verify order created in DB after UI checkout
String orderStatus = DatabaseUtils.getStringValue(
    "SELECT status FROM orders WHERE user_id='" + userId + "'
     ORDER BY created_at DESC LIMIT 1");
Assert.assertEquals(orderStatus, "PENDING");
```

12.6 Allure Reports Integration

Allure Reports provides beautiful, interactive test reports with steps, screenshots, attachments, and history trends. It is increasingly preferred over ExtentReports in modern frameworks.

```
<dependency>
  <groupId>io.qameta.allure</groupId>
  <artifactId>allure-testng</artifactId>
  <version>2.25.0</version>
</dependency>

// Annotate test methods for rich reporting
@Test
@Story("User Login")
@Description("Verify successful admin login and dashboard redirect")
@Severity(SeverityLevel.CRITICAL)
```

```
@Owner("john.doe@company.com")
@Link(name = "JIRA", url = "https://jira.example.com/TC-001")
public void testAdminLogin() {
    Allure.step("Navigate to login page", () -> {
        driver.get(baseUrl + "/login");
    });
    Allure.step("Enter credentials", () -> {
        loginPage.enterUsername("admin");
        loginPage.enterPassword("Admin@123");
    });
    Allure.step("Click login and verify dashboard", () -> {
        loginPage.clickLogin();
        Assert.assertTrue(dashboardPage.isLoaded());
    });
}

// Attach screenshot to Allure report
@Attachment(value = "Screenshot", type = "image/png")
public static byte[] captureScreenshot(WebDriver driver) {
    return ((TakesScreenshot) driver)
        .getScreenshotAs(OutputType.BYTES);
}
```

12.7 Test Naming Conventions & Best Practices

Aspect	Bad Practice	Best Practice
Test method naming	test1(), testLogin()	testSuccessfulLoginWithAdminCredentials()
Assert messages	Assert.assertTrue(condition)	Assert.assertTrue(condition, "Dashboard should be visible after login")
Test independence	Tests depend on execution order	Each test sets up and tears down its own data
Locator naming	By.id("e1")	private final By usernameInput = By.id("username")
Hard-coded waits	Thread.sleep(3000)	ExplicitWait with ExpectedConditions
Hard-coded URLs	driver.get("https://prod.com")	driver.get(ConfigReader.get("base.url"))
Test data in code	loginPage.enter("admin", "Admin@123")	Read from Excel/DataProvider/properties file
Assertions placement	Assertions inside Page classes	Assertions only in Test classes

12.8 GitHub Actions CI Alternative

For teams using GitHub, GitHub Actions provides a free, cloud-native CI/CD alternative to Jenkins with zero infrastructure management:

```
# .github/workflows/selenium-tests.yml
name: Selenium Test Suite

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 0 * * *'  # Nightly regression

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        browser: [chrome, firefox]

    steps:
    - uses: actions/checkout@v4

    - name: Set up JDK 21
      uses: actions/setup-java@v4
      with:
        java-version: '21'
        distribution: 'temurin'

    - name: Cache Maven packages
      uses: actions/cache@v4
      with:
        path: ~/.m2
        key: ${{ runner.os }}-m2-${{ hashFiles('**/pom.xml') }}

    - name: Run tests on ${{ matrix.browser }}
      run: |
        mvn test \
          -Dbrowser=${{ matrix.browser }} \
          -Dheadless=true \
          -Denv=staging
```

```
    - name: Upload test reports
      uses: actions/upload-artifact@v4
      if: always()
      with:
        name: test-report-${{ matrix.browser }}
        path: reports/
```

PRO TIP

Use GitHub Actions matrix strategy to run tests on multiple browsers simultaneously for free. Each browser runs in a separate parallel job, dramatically reducing total execution time.

Appendix: Quick Reference, Cheat Sheets & Resources

A. Complete pom.xml Reference

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.automation</groupId>
  <artifactId>selenium-framework</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <selenium.version>4.41.0</selenium.version>
    <testng.version>7.12.0</testng.version>
    <wdm.version>5.9.2</wdm.version>
    <extentreports.version>5.1.1</extentreports.version>
    <allure.version>2.25.0</allure.version>
    <log4j.version>2.23.0</log4j.version>
    <poi.version>5.3.0</poi.version>
    <restassured.version>5.4.0</restassured.version>
    <cucumber.version>7.15.0</cucumber.version>
    <jackson.version>2.17.1</jackson.version>
    <suiteFile>src/test/resources/testng.xml</suiteFile>
  </properties>

  <dependencies>
    <!-- Core Selenium -->
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>${selenium.version}</version>
    </dependency>
    <!-- WebDriverManager -->
    <dependency>
      <groupId>io.github.bonigarcia</groupId>
      <artifactId>webdrivermanager</artifactId>
      <version>${wdm.version}</version>
    </dependency>
    <!-- TestNG -->
    <dependency>
      <groupId>org.testng</groupId>
```

```
    <artifactId>testng</artifactId>
    <version>${testng.version}</version>
  </dependency>
  <!-- ExtentReports -->
  <dependency>
    <groupId>com.aventstack</groupId>
    <artifactId>extentreports</artifactId>
    <version>${extentreports.version}</version>
  </dependency>
  <!-- Allure TestNG -->
  <dependency>
    <groupId>io.qameta.allure</groupId>
    <artifactId>allure-testng</artifactId>
    <version>${allure.version}</version>
  </dependency>
  <!-- Log4j2 -->
  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-api</artifactId>
    <version>${log4j.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>${log4j.version}</version>
  </dependency>
  <!-- Apache POI -->
  <dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>${poi.version}</version>
  </dependency>
  <!-- REST Assured -->
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>${restassured.version}</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>json-schema-validator</artifactId>
    <version>${restassured.version}</version>
  </dependency>
  <!-- Cucumber -->
```

```
    <dependency>
      <groupId>io.cucumber</groupId>
      <artifactId>cucumber-java</artifactId>
      <version>${cucumber.version}</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>io.cucumber</groupId>
      <artifactId>cucumber-testng</artifactId>
      <version>${cucumber.version}</version>
      <scope>test</scope>
    </dependency>
    <!-- Jackson -->
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>${jackson.version}</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
        <configuration>
          <suiteXmlFiles>
            <suiteXmlFile>${suiteFile}</suiteXmlFile>
          </suiteXmlFiles>
          <argLine>-Xmx1024m -javaagent:
            ${settings.localRepository}/io/qameta/allure/
            allure-javaagent/${allure.version}/
            allure-javaagent-${allure.version}.jar
          </argLine>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

B. Selenium 4 Locator Cheat Sheet

Strategy	Syntax	When to Use
ID	By.id("submit")	Best — use when IDs are stable and unique
Name	By.name("username")	Form fields with name attributes
CSS Class	By.className("btn-primary")	Elements with unique, stable CSS classes
CSS Selector	By.cssSelector("#form input[type='text']")	Complex queries without traversing up DOM
XPath	By.xpath("//button[text()='Login']")	Text-based, parent traversal, complex axes
Link Text	By.linkText("Sign In")	Anchor tags with known exact text
Partial Link	By.partialLinkText("Sign")	Anchor tags with partial/variable text
Tag Name	By.tagName("input")	Collecting all elements of a type
Relative: below	with(By.tagName("input")).below(By.id("label"))	Element visually below another element
Relative: above	with(By.tagName("btn")).above(By.id("footer"))	Element visually above another element
Relative: toRightOf	with(By.tagName("input")).toRightOf(By.id("lbl"))	Form inputs to the right of their label

C. ExpectedConditions Quick Reference

Condition	Use Case
visibilityOfElementLocated(by)	Wait until element is visible on screen
elementToBeClickable(by)	Wait until element is visible AND enabled
presenceOfElementLocated(by)	Wait until element exists in DOM (may be hidden)
invisibilityOfElementLocated(by)	Wait for loading spinner / overlay to disappear
textToBePresentInElement(el, text)	Wait for dynamic text to appear in element
urlContains(text)	Wait for URL to change after navigation
titleContains(text)	Wait for page title to update
alertIsPresent()	Wait for JavaScript alert to appear
frameToBeAvailableAndSwitchToIt(by)	Wait for iframe and switch in one step
numberOfElementsToBeMoreThan(by, n)	Wait until list loads more than n items
attributeContains(el, attr, value)	Wait for element attribute to change

Condition	Use Case
stalenessOf(element)	Wait until previously found element becomes stale

D. TestNG Annotations Execution Order

```
Test Suite Execution Order:

1.  @BeforeSuite        (once, very first)
2.  @BeforeTest         (once per <test> tag in testng.xml)
3.  @BeforeGroups       (once before first test in the group)
4.  @BeforeClass        (once per test class)
5.  @BeforeMethod       (before EACH @Test method)
6.  @Test               (the actual test)
7.  @AfterMethod        (after EACH @Test method)
8.  @AfterClass         (once per test class)
9.  @AfterGroups        (once after last test in the group)
10. @AfterTest          (once per <test> tag in testng.xml)
11. @AfterSuite         (once, very last)

Note: alwaysRun=true ensures @After methods run even if @Test fails
```

E. REST Assured HTTP Status Codes Reference

Code	Status	Typical API Use Case
200	OK	Successful GET, PUT, PATCH response
201	Created	Successful POST — resource created
204	No Content	Successful DELETE — no body returned
400	Bad Request	Invalid request body, missing required fields
401	Unauthorized	Missing or invalid authentication token
403	Forbidden	Authenticated but not authorized for this resource
404	Not Found	Resource ID does not exist
409	Conflict	Duplicate resource — email already registered
422	Unprocessable	Validation error — invalid email format, etc.
429	Too Many Requests	Rate limiting triggered
500	Server Error	Unexpected server-side error
503	Service Unavailable	Server down for maintenance

F. Gherkin / Cucumber Quick Reference

Element	Example
Feature	Feature: Shopping Cart Management
Background	Background: Given the user is logged in as admin
Scenario	Scenario: Add item to empty cart
Scenario Outline	Scenario Outline: Add <quantity> of <product> to cart
Given (setup)	Given the cart is empty
When (action)	When the user adds "Blue T-Shirt" to the cart
Then (assert)	Then the cart should contain 1 item
And (chain)	And the cart total should display "$29.99"
But (negative)	But the wishlist should remain empty
Examples table	| product | quantity | price | expected_total |
@Tag	@smoke @cart @regression — used for selective execution
Doc String	For multi-line text arguments in steps
Data Table	For tabular data within a single step

G. Jenkins Pipeline Stages Reference

Pipeline Stage	Purpose	Key Commands
Checkout	Pull source code from Git	git branch: 'main', url: 'repo-url'
Build	Compile Java code	mvn clean compile -q
Smoke Tests	Fast validation on every commit	mvn test -DsuiteFile=smoke.xml
Regression Tests	Full suite on merge to main	mvn test -DsuiteFile=regression.xml
Reports	Publish HTML/Allure reports	publishHTML(), allure publish
Notifications	Email on pass/fail	emailext subject, body, to params
Cleanup	Remove workspace artifacts	cleanWs(), sh 'rm -rf screenshots/'

H. Common Java Exceptions in Selenium

Exception	Cause & Fix
NoSuchElementException	Element not found — check locator, add explicit wait, verify element exists in DOM

Exception	Cause & Fix
StaleElementReferenceException	Element re-rendered after finding — re-locate element, use retry logic
ElementNotInteractableException	Element hidden/disabled — scroll into view, wait for visibility/clickability
TimeoutException	Wait condition not met in time — increase timeout, check condition logic
ElementClickInterceptedException	Another element is overlaying — close overlay, scroll, use JS click as last resort
InvalidSelectorException	Malformed XPath/CSS — validate syntax in browser DevTools console
WebDriverException	Driver crashed or session expired — restart driver, check browser compatibility
NoSuchFrameException	Frame not found — wait for frame, verify frame ID/name
NoAlertPresentException	Alert not present — use explicit wait with alertIsPresent() condition
SessionNotCreatedException	Browser/driver version mismatch — update WebDriverManager or driver version

I. Recommended Learning Resources

Official Documentation

- Selenium Official Documentation — https://www.selenium.dev/documentation/
- TestNG Official Documentation — https://testng.org/doc/
- REST Assured Official Documentation — https://rest-assured.io/
- Jenkins Official Documentation — https://www.jenkins.io/doc/
- W3C WebDriver Standard Specification — https://www.w3.org/TR/webdriver/

Practice Applications for Automation

- Tecskool (Interactive Webelements) – https://practice.tecskool.com/
- Tecskool Ecommerce Website – https://testcart.tecskool.com/

J. Extended Glossary

Term	Definition
WebDriver	Interface defining browser automation methods in Selenium 4
POM	Page Object Model — design pattern separating page code from test logic

Term	Definition
TestNG	Test Next Generation — Java testing framework with rich annotations
DataProvider	TestNG annotation supplying parameterized test data sets
Explicit Wait	WebDriverWait targeting a specific element and condition
Fluent Wait	Explicit wait with custom polling interval and ignored exceptions
ThreadLocal	Java class providing thread-isolated variable instances — critical for parallel execution
Headless Browser	Browser running without visible GUI, used in CI/CD pipelines
CDP	Chrome DevTools Protocol — API for controlling Chrome at the browser engine level
Selenium Grid	Distributed test execution infrastructure for running tests on multiple machines/browsers
Jenkins Pipeline	Declarative CI/CD workflow defined as code in a Jenkinsfile
Surefire Plugin	Maven plugin that discovers and executes TestNG/JUnit test cases
SoftAssert	TestNG assertion mode that continues test execution after failures, reporting all at end
BDD	Behavior-Driven Development — describing tests in business language (Gherkin)
Gherkin	Plain English DSL used in Cucumber to write Feature files
Step Definition	Java method implementing a Gherkin step using @Given/@When/@Then
REST Assured	Java DSL for testing RESTful APIs with Given/When/Then syntax
POJO	Plain Old Java Object — simple class for serializing/deserializing API payloads
ExtentReports	Java library generating interactive HTML test execution reports
Allure	Open-source test reporting framework with rich UI and history trending
Singleton Pattern	Design pattern ensuring only one instance of a class (e.g., WebDriver) per thread
Factory Pattern	Design pattern that creates objects (e.g., browser driver) based on parameters
Builder Pattern	Design pattern for constructing complex objects with readable, fluent syntax
WebDriverManager	Library that auto-downloads and configures browser drivers (ChromeDriver, etc.)
Relative Locator	Selenium 4 locator finding elements by visual position relative to another element
IRetryAnalyzer	TestNG interface to retry failed tests automatically

Term	Definition
ITestListener	TestNG interface to hook into test lifecycle events
Apache POI	Java library for reading and writing Microsoft Office files (Excel, Word)

Thank You!

Mastering Java | *Selenium* | *TestNG* | *Jenkins* | *Framework Development*

Happy Testing! Build great automation frameworks and advance your career.